

AIQT CLI

AIQT Milestone 1 Build Specification v1.1

Canonical State Foundation

Build-planning document for implementing AIQT itself. Not a runtime document model for AIQT user projects.

Field	Value
Product	AIQT CLI
Specification	Milestone 1 Build Specification
Version	v1.1
Milestone	M1 - Canonical State Foundation
Aligns With	AIQT MVP Specification v0.6 and Technical Architecture Specification v0.3
Primary Agent Target	Claude Code or equivalent coding agent
Implementation Status	Ready for build handoff
Document score after clarification	95/100

Source Alignment

This document is derived from the AIQT MVP Specification v0.6 and the AIQT Technical Architecture Specification v0.3. The governing design decision is that AIQT behaves as a local workflow state engine, not as a markdown document generator. Runtime project milestones and work units are structured records, not persistent markdown files.

```
state -> graph -> packet -> checkpoint

not:
spec -> milestone file -> task file -> review file
```

1. v1.1 Clarification Summary

- Defines AssumptionSchema, QualityExpectationsSchema, and IntegrationsSchema.
- Clarifies camelCase as the only canonical JSON naming convention.
- Separates AIQT_SCHEMA_VERSION from package.json.version.
- Defines version compatibility exit codes.
- Clarifies that M1 init is non-interactive.
- Defines runlog corruption handling for status and next.

2. Milestone Objective

Milestone 1 builds the minimum working AIQT CLI foundation. It proves that AIQT can run as a local CLI, initialize a project in the current folder, create the lean canonical .aiqt/ file model, store durable project

context, store workflow state, append run log events, validate canonical state files, return deterministic human and JSON output, and report the next required workflow action.

Milestone 1 is intentionally narrow. It creates the engine floor, not the planning engine, agent packet engine, checkpoint engine, or export engine.

3. Commands in Scope

```
aiqt init
aiqt init --json
aiqt status
aiqt status --json
aiqt next
aiqt next --json
```

Optional non-interactive flags for init:

```
aiqt init --objective "<objective>"
aiqt init --target-user "<target user>"
aiqt init --agent "claude-code"
```

These flags make Milestone 1 testable without interactive prompts.

4. Out of Scope

- aiqt update, aiqt plan, aiqt checkpoint, aiqt review, aiqt export.
- Agent packet generation.
- Work graph generation, milestone generation, and work-unit generation.
- Checkpoint capture.
- Markdown exports.
- Claude Code automation or API integration.
- Project scaffolding.
- GitHub integration.
- AIQT.md, AGENTS.md, CLAUDE.md.
- Per-milestone markdown files or per-task markdown files.
- Interactive prompts in aiqt init.

5. Canonical Files Created by aiqt init

```
.aiqt/
  project.json
  state.json
  runlog.jsonl
  exports/
```

aiqt init must not create AIQT.md, AGENTS.md, CLAUDE.md, docs/, .milestones/, .tasks/, .aiqt/session.json, .aiqt/validation-history.json, or .aiqt/agent-actions.log.

6. Repository Implementation Scope

```
src/
```

```

index.ts
cli/
  register-commands.ts
  command-context.ts
  options.ts
  commands/
    init.command.ts
    status.command.ts
    next.command.ts
core/
  constants/
    schema-version.ts
  output/
    exit-codes.ts
    issue.ts
    result.ts
    human-output.ts
    json-output.ts
  filesystem/
    paths.ts
    file-exists.ts
    atomic-write.ts
    safe-writer.ts
    file-store.ts
  schema/
    common.schema.ts
    project.schema.ts
    state.schema.ts
    work-graph.schema.ts
    milestone.schema.ts
    work-unit.schema.ts
    dependency.schema.ts
    checkpoint.schema.ts
    runlog-event.schema.ts
  state/
    project-store.ts
    workflow-state-store.ts
    runlog-store.ts
    versioning.ts
    ids.ts
  workflow/
    statuses.ts
    actions.ts
    next-action.ts

tests/
  unit/
  integration/
  fixtures/
    empty-folder/
    initialized-project/
    corrupted-state/
    malformed-runlog/

```

7. Runtime and Package Setup

Area	Decision
Runtime	Node.js latest active LTS
Language	TypeScript
Package manager	pnpm
Module format	ESM
CLI parser	commander
Schema validation	zod
Testing	vitest

```

{
  "name": "aiqt",

```

```

"version": "0.5.0",
"type": "module",
"bin": {
  "aiqt": "./dist/index.js"
},
"scripts": {
  "dev": "tsx src/index.ts",
  "build": "tsc",
  "typecheck": "tsc --noEmit",
  "lint": "eslint .",
  "test": "vitest run",
  "test:watch": "vitest",
  "validate": "pnpm typecheck && pnpm lint && pnpm test"
}
}

```

8. Schema Version Rule

```

// src/core/constants/schema-version.ts
export const AIQT_SCHEMA_VERSION = "0.5.0" as const;

```

- project.json.version and state.json.version represent AIQT file schema version, not npm package version.
- package.json.version may later become 0.5.1 without changing AIQT_SCHEMA_VERSION.
- Version compatibility checks must compare canonical file versions against AIQT_SCHEMA_VERSION.

9. Data Contracts

9.1 Exit Codes

```

export const ExitCode = {
  Success: 0,
  ValidationFailed: 1,
  WorkflowBlocked: 2,
  InvalidInput: 3,
  MissingDependency: 4,
  ExternalIntegrationError: 5,
  HumanInputRequired: 10,
} as const;

```

Exit Code	M1 Use
0	Successful command
2	Workflow cannot proceed
3	Invalid command, invalid input, invalid/corrupt state, malformed canonical JSON, unsupported schema version
10	Human input required; not expected for M1 init because prompts are deferred

9.2 Issue Contract

```

export interface Issue {
  id: string;
  severity: "low" | "medium" | "high" | "critical";
  area: string;
  message: string;
  affectedItems?: string[];
}

```

```
suggestedAction?: string;
agentCanFix?: boolean;
}
```

9.3 Command Result Contract

```
export type CommandStatus =
  | "passed"
  | "failed"
  | "blocked"
  | "warning"
  | "needs_input";

export type WorkflowAction =
  | "init"
  | "update"
  | "plan"
  | "next"
  | "checkpoint"
  | "review"
  | "export"
  | "status";

export interface CommandResult<TData = unknown> {
  status: CommandStatus;
  action: WorkflowAction;
  projectStatus: ProjectStatus | null;
  currentMilestoneId: string | null;
  currentWorkUnitId: string | null;
  summary: string;
  completedActions: string[];
  changedFiles: string[];
  affectedItems: string[];
  blockingIssues: Issue[];
  warnings: Issue[];
  requiresHumanInput: boolean;
  nextRecommendedCommand: string | null;
  data?: TData;
  exitCode: number;
}
```

10. Schema Contracts

All canonical .aiqt/*.json files use camelCase field names. Snake_case examples from older MVP drafts are illustrative only and must not be used as file schema.

10.1 Project Model

```
export const ProjectModelSchema = z.object({
  version: z.string(),
  project: ProjectSchema,
  context: ProjectContextSchema,
  requirements: z.array(RequirementSchema),
  decisions: z.array(DecisionSchema),
  risks: z.array(RiskSchema),
  assumptions: z.array(AssumptionSchema),
  openQuestions: z.array(OpenQuestionSchema),
  quality: QualityExpectationsSchema,
  integrations: IntegrationsSchema.optional(),
});
```

10.2 Missing Schemas Resolved in v1.1

```
export const AssumptionSchema = z.object({
  id: z.string(),
  statement: z.string(),
```

```

    reason: z.string().nullable(),
    source: z.enum(["human", "aiqt", "agent", "system"]),
    status: z.enum(["active", "replaced", "invalidated"]),
    createdAt: z.string(),
    updatedAt: z.string(),
  });

export const QualityExpectationsSchema = z.object({
  acceptanceCriteriaRequired: z.boolean(),
  validationRequiredBeforeDone: z.boolean(),
  preferredValidationCommands: z.array(z.string()),
});

export const IntegrationsSchema = z.record(z.unknown());

```

10.3 Initial project.json after aiqt init

```

{
  "version": "0.5.0",
  "project": {
    "id": "PROJECT-001",
    "name": "current-folder-name",
    "objective": "",
    "targetUsers": [],
    "preferredAgent": null,
    "existingRepositoryPath": null,
    "createdAt": "ISO_DATE",
    "updatedAt": "ISO_DATE"
  },
  "context": {
    "constraints": [],
    "nonGoals": [],
    "technologyPreferences": [],
    "businessRules": [],
    "architectureNotes": []
  },
  "requirements": [],
  "decisions": [],
  "risks": [],
  "assumptions": [],
  "openQuestions": [],
  "quality": {
    "acceptanceCriteriaRequired": true,
    "validationRequiredBeforeDone": true,
    "preferredValidationCommands": []
  },
  "integrations": {}
}

```

10.4 Initial state.json after aiqt init

```

{
  "version": "0.5.0",
  "projectStatus": "draft",
  "currentMilestoneId": null,
  "currentWorkUnitId": null,
  "workGraph": {
    "milestones": [],
    "workUnits": [],
    "dependencies": []
  },
  "checkpoints": [],
  "lastAgentPacket": null,
  "nextRecommendedCommand": "aiqt update",
  "lastUpdatedAt": "ISO_DATE"
}

```

10.5 Runlog Event

```

export const RunlogEventSchema = z.object({

```

```

id: z.string(),
type: z.string(),
timestamp: z.string(),
actor: z.enum(["human", "aiqt", "agent", "system"]),
summary: z.string(),
relatedIds: z.array(z.string()),
data: z.record(z.unknown()).optional(),
});

```

```

{"id":"EVT-001","type":"project.initialized","timestamp":"ISO_DATE","actor":"aiqt","summary":"Project initialized with canonical AIQT state files.","relatedIds":["PROJECT-001"],"data":{"schemaVersion":"0.5.0"}}

```

11. aiqt init Behavior

Purpose: Initialize a new AIQT project in the current folder.

1. Verify the current folder is writable.
2. Verify .aiqt/ does not already exist.
3. Derive project name from the current folder.
4. Apply optional non-interactive flags: --objective, --target-user, --agent.
5. Create .aiqt/.
6. Create .aiqt/project.json.
7. Create .aiqt/state.json.
8. Create .aiqt/runlog.jsonl.
9. Create .aiqt/exports/.
10. Append project.initialized to runlog.jsonl.
11. Return nextRecommendedCommand = aiqt update.

Blocked Case	Exit Code
.aiqt/ already exists	3
current folder not writable	3
malformed existing .aiqt/ directory	3
file write failure	3

11.1 Init Interaction Rule

- Milestone 1 does not implement interactive prompts.
- If --objective is omitted, objective remains empty.
- If --target-user is omitted, targetUsers remains empty.
- If --agent is omitted, preferredAgent remains null.
- Interactive project intake is deferred to Milestone 2 through aiqt update.

12. aiqt status Behavior

Purpose: Inspect current AIQT state without modifying files.

12. Locate .aiqt/.
13. Read .aiqt/project.json.

14. Read .aiqt/state.json.
15. Validate schemas and versions.
16. Inspect runlog.jsonl health.
17. Return project status, current milestone, current work unit, counts, blockers, warnings, and next command.
18. Do not modify project.json, state.json, runlog.jsonl, or exports.

Human-readable output should include project name, project status, milestone count, work-unit count by status, current milestone, current work unit, warnings, blocking issues, and next recommended command.

13. aiqt next Behavior

Purpose: In Milestone 1, report the next required workflow action. Full agent packet generation is deferred to a later milestone.

19. Locate .aiqt/.
20. Read project.json.
21. Read state.json.
22. Validate schemas and versions.
23. Inspect runlog.jsonl health.
24. Calculate next required workflow action.
25. Return next command and reason.
26. Do not mutate state in Milestone 1.

Condition	Result
.aiqt/ missing	Block: run aiqt init
project is draft and context is incomplete	Recommend aiqt update
project has context but no work graph	Recommend aiqt plan
work graph exists but packet generation not implemented yet	Warn that full aiqt next arrives later

14. Runlog Health Rules

Case	Behavior	Exit Code
missing runlog.jsonl	Invalid state	3
unreadable runlog.jsonl	Invalid state	3
malformed individual line	Warning, do not block status/next	0 if project/state valid
project.json or state.json malformed	Invalid state	3

Malformed individual runlog lines must appear in warnings with severity medium, area runlog, and should be exposed in JSON output under data.runlogHealth.

```
"data": {
  "runlogHealth": {
    "totalLines": 3,
    "validLines": 2,
    "malformedLines": 1,
    "malformedLineNumbers": [3]
  }
}
```

}

15. Safe Write Rules

- Serialize JSON with stable formatting.
- Write temporary file beside target file.
- Flush file content.
- Replace target file atomically.
- Remove temporary file on success.
- Avoid leaving partial JSON on failure.

Existing File	Command	Behavior
.aiqt/ exists	aiqt init	Block
canonical file exists	aiqt init	Block
malformed .aiqt/ exists	aiqt init	Block
missing canonical file in existing project	status/next	Report invalid state
malformed canonical JSON	status/next	Report invalid state

No --force behavior exists in Milestone 1.

16. Version Compatibility

File Version	Behavior	Exit Code
0.5.0	Proceed	0
older compatible version	Proceed only if schema validates	0 or 3
future unsupported version	Block	3
missing version	Block	3
invalid version type	Block	3

```
AIQT cannot continue.
Reason:
This project was created with a newer AIQT version.
Suggested action:
Upgrade AIQT to the latest version.
```

17. Implementation Tasks

Task	Scope	Acceptance
M1-T1	Project Scaffold	pnpm install, pnpm build, pnpm test, and pnpm validate work.
M1-T2	CLI Registration	help output works and unknown commands exit with code 3.
M1-T3	Output Contract	all handlers return CommandResult and --json emits stable JSON.
M1-T4	Schemas	valid initial project/state pass and invalid statuses fail.
M1-T5	File Store and Atomic Writes	JSON writes are atomic and failed

		writes do not leave partial target files.
M1-T6	State Stores	stores validate schemas, future versions block, IDs increment by prefix.
M1-T7	aiqt init	creates only canonical files, appends project.initialized, blocks if .aiqt/ exists.
M1-T8	aiqt status	reads and validates state, shows counts and next command, no mutation.
M1-T9	aiqt next	after init recommends aiqt update; context/no graph recommends aiqt plan.
M1-T10	Test Suite	schema, version, atomic write, init/status/next, corrupted state, runlog tests pass.

18. Milestone 1 Definition of Done

- CLI can be executed as aiqt.
- aiqt init creates the lean canonical state model.
- No markdown docs are created by default.
- No milestone or task files are created.
- project.json stores durable project context.
- state.json stores workflow state and an empty work graph.
- runlog.jsonl stores append-only initialization event.
- status reads state without mutation.
- next reports the next required workflow action without mutation.
- JSON output is deterministic.
- Exit codes are consistent.
- Invalid state is reported clearly.
- Future versions are blocked with exit code 3.
- Malformed runlog lines warn without blocking status/next when project/state are valid.
- Core behavior is tested.
- The project is ready for Milestone 2: aiqt update.

19. Milestone 1 Agent Execution Packet

```
# AGENT EXECUTION PACKET
## Role
You are implementing Milestone 1 of AIQT CLI.
AIQT is a local TypeScript CLI workflow state engine for projects built with AI coding agents.

## Current Work Unit
ID: AIQT-M1
Title: Canonical State Foundation
Objective: Build the minimum CLI foundation that can initialize, inspect, and navigate a lean `.aiqt/`
project state.

## Scope
```

Implement:

- TypeScript CLI executable named `aiqt`
- `aiqt init`, `aiqt init --json`
- `aiqt status`, `aiqt status --json`
- `aiqt next`, `aiqt next --json`
- optional init flags: `--objective`, `--target-user`, `--agent`
- canonical `.aiqt/` file creation
- project/state/runlog schemas and stores
- command result contract, issue schema, exit codes
- human output renderer and JSON output renderer
- atomic JSON writes
- AIQT_SCHEMA_VERSION compatibility checks
- runlog health warnings
- unit and integration tests

Out of Scope**Do not implement:**

- `aiqt update`, `aiqt plan`, `aiqt checkpoint`, `aiqt review`, `aiqt export`
- interactive prompts in `aiqt init`
- agent packet generation
- work graph generation
- milestone or work-unit generation
- markdown documentation generation
- `AIQT.md`, `AGENTS.md`, `CLAUDE.md`
- `.aiqt/session.json`, `.aiqt/validation-history.json`, `.aiqt/agent-actions.log`
- GitHub integration
- Claude Code API integration

Required Runtime

- Node.js latest active LTS
- TypeScript
- pnpm
- ESM
- commander
- zod
- vitest

Acceptance Criteria

- `pnpm build` passes.
- `pnpm test` passes.
- `pnpm validate` passes.
- `aiqt init` creates only the canonical files.
- `aiqt init` blocks if `.aiqt/` already exists.
- `aiqt status` reads state and does not modify files.
- `aiqt next` reports the next required command and does not mutate state.
- `--json` output is deterministic and matches the command result contract.
- invalid JSON state returns a clear error with exit code 3.
- unsupported future versions are blocked with exit code 3.
- malformed runlog lines produce warnings, not blocking errors, when project/state are valid.
- `runlog.jsonl` contains a valid `project.initialized` event.

Validation Commands

```
pnpm typecheck
pnpm lint
pnpm test
pnpm build
pnpm validate
```

Required Agent Output**Return:**

1. Summary of implementation.
2. Files created.
3. Files modified.
4. Tests added.
5. Validation commands run.
6. Any unresolved issues.
7. Suggested next step.

Appendix A - Minimum Test Matrix

Test Type	Scenario	Expected Result
Unit	project schema validates initial model	Pass
Unit	state schema validates initial model	Pass
Unit	runlog schema validates initialized event	Pass
Unit	future version blocks	Exit code 3
Unit	missing version blocks	Exit code 3
Unit	ID generation increments by prefix	Stable next ID
Unit	atomic write writes valid JSON	No partial file
Integration	aiqt init creates canonical files	Only .aiqt files created
Integration	aiqt init --json returns valid JSON	CommandResult shape
Integration	aiqt init with flags stores values	objective/targetUsers/preferredAgent set
Integration	aiqt init blocks if .aiqt exists	Exit code 3
Integration	aiqt status works after init	Draft status
Integration	aiqt next after init recommends update	nextRecommendedCommand = aiqt update
Integration	corrupted project.json causes status failure	Exit code 3
Integration	corrupted state.json causes status failure	Exit code 3
Integration	malformed runlog line causes warning	Exit code 0 if project/state are valid
Integration	runlog contains project.initialized event	Valid JSONL event